

Exception Handling

1. Basic Try-Catch

Write a Java program that takes two integers as input and prints their quotient. Handle

`ArithmeticException` if the denominator is zero.

2. Multiple Catch Blocks

Write a program to read an element from an integer array by index. Handle both `ArrayIndexOutOfBoundsException` and `InputMismatchException`.

3. Using `finally` Block

Write a program that opens a file (you can simulate it), reads a line, and always prints

"Closing resources" using a `finally` block.

4. Throwing Exceptions

Write a method `checkAge(int age)` that throws an `IllegalArgumentException` if the age is

less than 18. Catch the exception in the `main()` method.

5. Using `throws` Keyword

Write a method `readFile(String filename)` that declares `throws IOException`. Call this

method from `main()` and handle the exception there.

6. Custom Exception Class

Create a custom exception called `InvalidEmailException`. Write a method `validateEmail(String email)` that throws this exception if the email doesn't contain `@`. Handle it in `main()`.

7. Nested Try-Catch

Write a program using nested `try-catch` blocks where an inner `try` block handles an `ArithmeticException`, and the outer handles any other exception.

8. Re-throwing Exceptions

Write a method that catches an exception, logs it, and then re-throws it to be handled in the calling method.

9. Try with Multiple Resources

Use Java's try-with-resources to read from a file using `BufferedReader`. Handle any exceptions that occur.

10. Handling Checked and Unchecked Exceptions

Write a program demonstrating the difference between checked (`IOException`) and unchecked (`NumberFormatException`) exceptions by creating situations where each is thrown and handled.

11. You are tasked with creating a **Student Management System** in Java. The system should store student details and perform basic validations and file operations. The system must be **robust**, with complete exception handling using Java's mechanisms.

Requirements:

Create a class named `Student` with the following fields:

- `int id`
- `String name`
- `int age`
- `double grade`

Create the following classes and components:

1. Custom Exceptions:

Define two custom exception classes:

- `InvalidAgeException` – Thrown when age is less than 5 or more than 100.
- `InvalidGradeException` – Thrown when grade is less than 0.0 or more than 100.0.

2. Student Constructor with Validation:

- Inside the constructor of `Student`, validate `age` and `grade`.
- If invalid, **throw** the appropriate custom exception using the `throw` keyword.

3. Method to Save Student to File:

- Create a method `saveToFile()` that writes student data to a file (`students.txt`).
- Use **try-with-resources** to handle file operations using `FileWriter`.
- If any IO error occurs, catch it using appropriate `catch` blocks.

4. Main Method (or Tester Class):

- In the `main` method:
 - Use a **try-catch-finally** block to create a new `Student` object from user input.
 - Use multiple `catch` blocks to catch:
 - `InvalidAgeException`
 - `InvalidGradeException`
 - `InputMismatchException`
 - `IOException`
 - A generic `Exception` for fallback.
 - In the `finally` block, display: "End of student registration process."

Additional Instructions:

- Validate all user inputs (e.g., name cannot be null or empty).
- You may use `Scanner` for user input.
- Ensure exception messages are clear and helpful.
- Explain:
 - When and why each exception is thrown.
 - How `throw` is used to raise an exception manually.
 - How `throws` is used in method signatures.
 - The purpose of `finally`.
 - How **try-with-resources** helps with resource management.

Sample Output

```
Enter student ID: 101
Enter student name: Alice
Enter age: 3
InvalidAgeException: Age must be between 5 and 100.
End of student registration process.
```